

XV EDIÇÃO



MARATONA DE PROGRAMAÇÃO

InterFatecs

1a. Fase

23 de maio de 2026

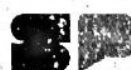
CADERNO DE QUESTÕES

ORGANIZAÇÃO E APOIO

Fatec

Garça
Dep. Júlio Lúcio
Marcondes de Moura

CPS
Centro
Paula Souza



**SÃO
PAULO**

GOVERNO
DO ESTADO
SÃO PAULO SÃO PAULO

WWW.INTERFATECS.COM.BR

1 Instruções

Este caderno contém 11 problemas – identificados por letras de A até K, com páginas numeradas de 3 até 27. Verifique se seu caderno está completo.

Informações gerais

1. Sobre a competição

- (a) A competição possui duração de 5 horas (início às 13:00h término às 18:00h);
- (b) NÃO é permitido acesso a conteúdo da Internet ou qualquer outro meio eletrônico digital;
- (c) NÃO é permitido o uso de ferramentas de auxílio à codificação, como GitHub Copilot, Tabnine, Amazon CodeWhisperer ou similar;
- (d) É permitido somente acesso a conteúdo impresso em papel (cadernos, apostilas, livros);
- (e) Não é permitida a comunicação com o técnico ou qualquer outra pessoa que não seja a equipe para tirar dúvidas sobre a maratona
- (f) Cada equipe terá acesso a 1 computador dotado do ambiente de submissão de programas (BOCA), dos compiladores, link-editores e IDEs requeridos pelas linguagens de programação permitidas;
- (g) NÃO é permitido o uso de notebooks, smartphones, ou outro tipo de computador ou assistente pessoal;
- (h) Todos os problemas têm o mesmo valor na correção.

2. Sobre o arquivo de solução e submissão:

- (a) O arquivo de solução (o programa fonte) DEVE TER o mesmo nome que o especificado no enunciado (logo após o título do problema);
- (b) confirme se você escolheu a linguagem correta e está com o nome de arquivo correto antes de submeter a sua solução;
- (c) NÃO insira acentos ou outros caracteres especiais no arquivo-fonte.

3. Sobre a entrada

- (a) A entrada de seu programa deve ser lida da entrada padrão (não use interface gráfica);
- (b) Seu programa será testado em vários casos de teste válidos além daqueles apresentados nos exemplos. Considere que seu programa será executado uma vez para cada caso de teste.

4. Sobre a saída

- (a) A saída do seu programa deve ser escrita na saída padrão;
- (b) Não exiba qualquer outra mensagem além do especificado no enunciado.

5. Versões das linguagens

- (a) gcc version 9.4.0 (lembre-se que não existem bibliotecas do Windows)
- (b) Python 3.8.10
- (c) Java 11.0.22 (lembre-se que as classes devem estar fora de pacotes)

Problem A

Basketball

Source file: basketball.{ c | cpp | java | py }

Author: Prof. Dr. Alex Marino (Fatec Ourinhos)

Little Johnny, our intrepid friend, likes to spend his free time watching basketball practices in the sumptuous gymnasium of the Esbornian capital. While his friends argue about who will score the most baskets, Johnny's programmer spirit makes him notice something different: the trajectory of the ball as it falls to the ground.

He noticed that whenever the ball is dropped from a height H , it hits the ground and bounces. With each new bounce, the ball reaches a height that is a fraction F of the previous height. Johnny realized that the ball travels a considerable distance while bouncing repeatedly.

Your task is to help Johnny calculate the total distance traveled by the ball (going up and down) until the moment it touches the ground for the N -th time.

Input

A real value H ($1.0 \leq H \leq 1000.0$), representing the initial height from which the ball is dropped.

A real value F ($0.0 < F < 1.0$), representing the fraction of the height that the ball recovers after each bounce.

An integer value N ($1 \leq N \leq 100$), representing the bounce number (ground touch) that Johnny wants to observe.

Output

Print a single line containing the phrase "Total distance: " followed by the value of the distance traveled, with two decimal places.

Example of Input 1

10.0 0.5 2

Example of Output 1

Total distance: 20.00

Example of Input 2

5.0 0.2 3

Example of Output 2

Total distance: 7.40

Example of Input 3

100.0 0.1 1

Example of Output 3

Total distance: 100.00

Problema B

ChatMunarinho

Arquivo fonte: chatmunarinho.{ c | cpp | java | py }
Autor: Prof. Dr. Leandro Luque (Fatec Mogi das Cruzes/U-TAD Madrid)

Munarinho gosta muito de História, Política e afins. Como seu amigo Jeff não está mais compartilhando dos mesmos gostos, ele vem conversando com LLMs sobre o assunto. Porém, no mês passado, a conta de Munarinho veio muito alta e ele finalmente decidiu implementar sua própria LLM, o ChatMunarinho.

Atualmente, ele está na fase de tokenização — processo de dividir a entrada em grupos de caracteres para depois convertê-los em números (o que um computador realmente entende).

Determinando que bibliotecas prontas são muito fáceis, ele decide implementar sua própria versão do algoritmo *Byte Pair Encoding* (BPE). Porém, como bom inventor de problemas elegantes, ele impõe uma modificação curiosa:

- Em vez de pares, o algoritmo deve considerar **trios consecutivos de tokens**. ✓

Agora, Munarinho quer simular esse processo e entender como sua tokenização evolui ao longo do tempo.

* string = frase

Definição do algoritmo

O processo funciona assim:

→ vocabulário = frase

1. Inicialize o vocabulário com todos os caracteres distintos da string de entrada (incluindo espaços, vírgulas e pontos).
2. Separe a string em uma lista de tokens (inicialmente, caracteres individuais).
→ = split()
3. Enquanto o tamanho do vocabulário for menor que o limite máximo: ✓ < LM
 - (a) Conte todas as ocorrências de **trios consecutivos** de tokens.
 - (b) Escolha o **trio mais frequente**.
 - i. Se nenhum trio aparecer mais de uma vez, interrompa — exceto se restar apenas um trio possível. Neste caso, continue.
 - ii. Em caso de empate, escolha o trio que aparece primeiro se ordenado alfabeticamente.
 - (c) Crie um novo token com o trio. ✓
 - (d) Substitua todas as ocorrências desse trio na sequência atual (da esquerda para a direita, sem sobreposição).
 - (e) Adicione o novo token ao vocabulário. ✓

Exemplo ilustrativo (passo-a-passo)

Considere a string:

abababa

E limite de vocabulário $V = 10$.

Estado inicial:

- Tokens: a b a b a b a
- Vocabulário: a, b

Iteração 1

Trios e frequências:

- (a, b, a): 3
- (b, a, b): 2

Escolhido: (a, b, a)

Novo token: aba

Nova sequência:

aba b aba

Vocabulário: a, b, aba

→ + = vocabulário

Iteração 2

Trios:

- (aba, b, aba): 1

Escolhido: (aba, b, aba)

Novo token: abababa

Nova sequência:

abababa

Vocabulário: a, b, aba, abababa

Iteração 3

Não há mais trios possíveis.

Resultado final:

abababa

Entrada

A entrada contém:

- Uma string S ($1 \leq |S| \leq 10^5$), composta potencialmente por caracteres minúsculos, espaços, vírgulas e pontos.
- Um inteiro V ($1 \leq V \leq 1000$), indicando o tamanho máximo do vocabulário.

Saída

Imprima a entrada conforme formatada após o processo de tokenização.

Observações

- A substituição dos trios deve respeitar a ordem da esquerda para a direita, sem sobreposição.
- Após cada adição ao vocabulário, os trios devem ser recalculados com base na nova sequência.
- É garantido que o algoritmo termina.
- Atenção à eficiência: soluções ingênuas podem não passar no tempo limite.

Exemplo de Entrada 1

abcabcabc

1

Exemplo de Saída 1

a b c a b c a b c

Exemplo de Entrada 2

abcabcabc

4

Exemplo de Saída 2

abc abc abc

Exemplo de Entrada 3

```

abccababc
5
  
```

Exemplo de Saída 3

```

abccababc
  
```

Exemplo de Entrada 4

```

abccababc abc
6
  
```

Exemplo de Saída 4

```

abccababc
  
```

```

VOCABULARIO = input()
string = input()
tokens = [" "]
tokens = string.split
while string.length < VOCABULARIO:
    tokens = string.split
    ... continue = token
  
```

Problema C

Ciclos de Esforço

Arquivo fonte: ciclo.{ c | cpp | java | py }
 Autor: Prof. Dr. Alex Marino (Fatec Ourinhos)

Joãozinho nosso sagaz amigo, agora professor da Faculdade de Tecnologia da Esbórnia, desenvolveu a Metodologia dos Ciclos de Esforço (MCE) para preparar seus alunos para o Maratona de Programação da Esbórnia de 2026.

A regra é clara: cada Ciclo de Esforço dura exatamente K minutos de codificação intensa. No entanto, nem sempre os problemas são resolvidos no tempo exato de um ciclo. O tempo excedente após o último ciclo completo é denominado Minutos de Refatoração.

Joãozinho deseja consolidar o tempo total de treino de suas equipes.

Entrada

A primeira linha da entrada contém dois inteiros:

- N e K , onde: $1 \leq N \leq 100$ representa o número de dias de treinamento registrados.
- $1 \leq K \leq 1440$ representa a duração, em minutos, de um único Ciclo de Esforço.

As N linhas seguintes contêm, cada uma, dois inteiros C e M , onde:

- C é o número de ciclos completos realizados naquele dia.
- M é a quantidade de minutos de refatoração despendidos no mesmo dia.

Saída

Relatorio MCE: X minutos de esforco total, onde X deve ser substituído pelo valor da soma total do tempo de dedicação.

Exemplo de Entrada 1

```
2 50
3 10
2 5
```

Exemplo de Saída 1

```
Relatorio MCE: 265 minutos de esforco total
```

Exemplo de Entrada 2

```
1 60
1 0
```

Exemplo de Saída 2

```
Relatorio MCE: 60 minutos de esforco total
```

Exemplo de Entrada 3

```
3 45
0 10
0 20
0 5
```

Exemplo de Saída 3

```
Relatorio MCE: 35 minutos de esforco total
```


Problema E

Eco da Palavra

Arquivo fonte: eco.{ c | cpp | java | py }

Autor: Prof. Dr. Lucas Baggio Figueira (Fatec Ribeirão Preto)

Bilbo encontrou um pergaminho antigo onde está escrita uma palavra misteriosa. Segundo Gandalf, algumas palavras possuem eco mágico: uma palavra tem eco se seu primeiro e seu último caractere forem iguais, bem como o segundo e o penúltimo, o terceiro e o antepenúltimo, e assim por diante.

Sua tarefa é ajudar Bilbo a verificar se a palavra possui eco mágico.

Entrada

A entrada contém uma única linha com uma palavra S .

- A palavra contém apenas letras minúsculas (a-z).
- $1 \leq |S| \leq 1000$

Saída

Imprimir SIM se a palavra possui eco mágico, NAO caso contrário.

Exemplo de Entrada 1

abca	NAO
------	-----

Exemplo de Saída 1

Exemplo de Entrada 2

radar	SIM
-------	-----

Exemplo de Saída 2

```
tot tempo = 0      tot nota = 0
P, N = input().split()
S = input()
l, D = len(S), S
for i in range(1, D):
    if (tot tempo > tempo):
        break -> ou print tot tempo
    else:
        tot tempo += 1
        tot nota += 1
```

Problema F

Fotos do Rolo

Arquivo fonte: fotos.{ c | cpp | java | py }

Autor: Prof. Dr. Lucas Baggio Figueira (Fatec Ribeirão Preto)

Em um totem automático de impressão de fotos, os clientes escolhem diferentes tamanhos de impressão para suas imagens. O totem utiliza um rolo de papel fotográfico com largura fixa de 10 cm.

Cada serviço de impressão é descrito por uma dimensão no formato AxB (por exemplo 10x15, 15x10, 10x20). A descrição pode aparecer em qualquer ordem, porém uma das dimensões será sempre 10, pois corresponde à largura fixa do papel.

Assim, o consumo do rolo depende apenas da outra dimensão, que representa o comprimento da foto em centímetros.

Por exemplo:

Descrição	Comprimento consumido
10x15	15 cm
15x10	15 cm
10x20	20 cm

Durante um atendimento, um cliente solicita várias cópias de diferentes serviços. Entretanto, o rolo de papel restante no totem pode não ser suficiente para imprimir todas as fotos.

O sistema do totem deve decidir quais fotos imprimir de modo que:

1. O consumo total de papel não ultrapasse o papel disponível.
2. O valor total cobrado pelas fotos impressas seja máximo possível.

Cada cópia de uma foto pode ser impressa ou descartada independentemente das demais.

Sua tarefa é determinar se o pedido pode ser atendido completamente. Caso não seja possível, determine o maior valor total possível que o totem pode cobrar imprimindo apenas parte das fotos.

Entrada

A primeira linha contém dois inteiros:

N R

onde:

- N é o número de tipos de serviço solicitados ($1 \leq N \leq 100$)
- R é a quantidade de papel restante no rolo, em metros ($1 \leq R \leq 100$)

Cada uma das próximas N linhas descreve um serviço no formato:

descricao quantidade valor

onde:

- descricao é uma string no formato AxB
- quantidade é o número de cópias solicitadas ($1 \leq quantidade \leq 1000$)
- valor é o valor unitário da impressão ($1 \leq valor \leq 1000$)

ENTRADA ÚNICA

É garantido que em toda descrição uma das dimensões será exatamente 10.

Saída

Imprima uma única linha no formato:

COMPLETO V

se todas as cópias puderem ser impressas, ou

PARCIAL V

se for necessário remover algumas cópias para respeitar o limite do rolo.

Em ambos os casos, V deve representar o valor total máximo possível obtido com as fotos impressas.

Exemplo de Entrada 1

3 2
10x15 5 4
20x10 3 7
10x30 2 10

Exemplo de Saída 1

COMPLETO 61

Exemplo de Entrada 2

3 1
10x15 4 5
10x20 3 6
25x10 2 8

20
 $\begin{matrix} 18 \\ 16 \end{matrix} = 32$

Exemplo de Saída 2

PARCIAL 32

$N, B = \text{INT}(\text{INPUT}())$
descricao, quantidade, valor

descricao = "

Problema G

Ressonância

Arquivo fonte: `ressonancia.{ c | cpp | java | py }`
Autor: Prof. Dr. Alex Marino (Fatec Ourinhos)

O serviço secreto da Monetland detectou uma incursão massiva: a DarioLandia lançou uma nuvem de N micro-drones espíões de última geração. No entanto, o Presidente Big Orange recebeu uma informação técnica vital do agente Laranjaponga: esses drones utilizam um sistema de comunicação por Sincronia de Fase Quântica.

Essa tecnologia possui uma vulnerabilidade catastrófica: quando dois drones operam a uma distância muito curta entre si, seus campos eletromagnéticos sofrem uma Ressonância Destrutiva. Se o Laranjaponga conseguir disparar um pulso de micro-ondas exatamente no ponto onde dois drones estão mais próximos, a interferência gerada por esse par causará uma reação em cadeia, desativando toda a rede neural da nuvem de drones instantaneamente.

O tempo é curto. Os drones estão se movendo para posições de ataque. Ajude Joãozinho, ops agente Laranjaponga, identificar essa vulnerabilidade calculando a distância mínima absoluta entre qualquer par de drones na nuvem invasora. Se você falhar em encontrar o par mais próximo rapidamente, os radares da DarioLandia travarão os drones da Monetland e o mundo civilizado cairá.

Entrada

Um inteiro N ($2 \leq N \leq 10^5$), representando a quantidade de drones detectados. N linhas, cada uma contendo dois inteiros X_i e Y_i ($-10^9 \leq X_i, Y_i \leq 10^9$), as coordenadas espaciais de cada drone no plano de defesa.

Saída

Imprima um único número real com exatamente 4 casas decimais, representando a menor distância encontrada entre dois drones quaisquer.

Exemplo de Entrada 1

3	5.0000
10 50	
100 50	
15 50	

Exemplo de Saída 1

Exemplo de Entrada 2

3	1.0000
50 10	
50 100	
50 11	

Exemplo de Saída 2

Problema H

Segredos de Munarinho

Arquivo fonte: segredos.{ c | cpp | java | py }

Autor: Prof. Dr. Leandro Luque (Fatec Mogi das Cruzes/U-TAD Madrid)

Munarinho e seu amigo Jeff estão trocando cartas para escapar da vigilância de terceiros curiosos, que andaram extraviando alguns envelopes. Nessa troca-troca secreto deles, decidiram inventar um idioma cifrado onde cada palavra é apenas um codinome para outra palavra real.

O detalhe é que Munarinho adora testar o raciocínio de Jeff e, em vez de revelar diretamente o significado de cada codinome, ele envia pistas dizendo o que a palavra **eh** ou o que ela **nao eh**. Jeff, que já perdeu a paciência algumas vezes tentando entender uma única frase, pediu sua ajuda para desenvolver um programa que decifre a mensagem cifrada.

Cada codinome possui um conjunto inicial de possíveis significados. As pistas fornecidas por Munarinho restringem essas possibilidades até que reste exatamente uma palavra válida para cada codinome.

Sua missão é decifrar completamente a mensagem secreta e traduzi-la para o vocabulário original.

Entrada

A entrada é composta por quatro partes:

- Um inteiro N ($1 \leq N \leq 100$), indicando o número de codinomes do idioma.
- N linhas contendo pistas. Cada linha corresponde a um codinome e possui uma ou mais pistas no formato:

- X eh Y; ou X nao eh Y;

As pistas de uma mesma linha são separadas por ponto e vírgula ;.

- N linhas indicando os possíveis significados (podem ser especificados até $1000N$ significados) de cada codinome, no formato:

- X:A;B;C;...

- Uma linha final contendo a mensagem secreta, composta por codinomes separados por espaço.

Todos os codinomes e significados são formados apenas por letras maiúsculas e/ou minúsculas, sem acentos, dígitos ou caracteres especiais.

Saída

Imprima a mensagem decifrada, substituindo cada codinome pelo seu significado único determinado após aplicar todas as pistas.

Observações



- É garantido que cada codinome terá exatamente uma solução válida após aplicar todas as pistas.
- Nenhuma pista será contraditória.
- Todos os codinomes usados na mensagem estarão definidos no conjunto.

Exemplo de Entrada 1

```
3
Alfa nao eh Cachorro;Alfa nao eh Gato
Beta eh com;Beta nao eh Leao
Gama nao eh Vaca;Gama nao eh Boi
Alfa:Cachorro;Gato;Estou
Beta:Leao;com;Rinoceronte
Gama:Vaca;Boi;saudades
Alfa Beta Gama
```

Exemplo de Saída 1

```
Estou com saudades
```

Exemplo de Entrada 2

```
1
bola nao eh bingo;bola nao eh cadeira;bola nao eh bola
bola:bola;bingo;cadeira;ha
bola bola bola
```

Exemplo de Saída 2

```
ha ha ha
```


1 2 3



Problema I

Brinquedo Educativo

Arquivo fonte: brinquedo.{ c | cpp | java | py }

Autor: Prof. Dr. Reinaldo Arakaki (Fatec São José dos Campos)

MF
0
1 2 3 ...
K

Mariazinha ganhou um brinquedo educativo para praticar matemática. O brinquedo é formado por blocos numerados sequencialmente de 1 até N .

A brincadeira consiste em dividir todos os blocos em exatamente K conjuntos, de forma que:

- cada bloco pertença a apenas um conjunto;
- nenhum bloco fique de fora;
- a soma dos valores dos blocos seja igual em todos os conjuntos. •

3 2 11
1, 2, 3, 4, 5, 6

2

Sua tarefa é ajudar Mariazinha a descobrir se essa divisão é possível.

Entrada

A primeira linha da entrada contém dois inteiros:

30 1-6 7-12
5

- N e K , onde: $1 \leq N \leq 100$ quantidade de blocos numerados de 1 até N .
- $1 \leq K \leq 100$ quantidade de conjuntos desejados.

Saída

Caso seja possível dividir os blocos em ' K ' conjuntos de mesma soma, imprima: POSSIVEL

Caso contrário: IMPOSSIVEL

Exemplo de Entrada 1

5 2

Exemplo de Saída 1

IMPOSSIVEL

Exemplo de Entrada 2

6 2

Exemplo de Saída 2

IMPOSSIVEL

Exemplo de Entrada 3

30 5

Exemplo de Saída 3

POSSIVEL

SAÍDA
 $N \% K = 0 \rightarrow$ POSSIVEL
 $N \% K \neq 0 \rightarrow$ IMPOSSIVEL

Problema J

Parque de Diversões

Arquivo fonte: `parque.{ c | cpp | java | py }`

Autor: Prof. Dr. Reinaldo Arakaki (Fatec São José dos Campos)

Marina é apaixonada por parques de diversão e decidiu passar o dia em um grande parque temático. O parque funciona durante T horas, e cada brinquedo possui um tempo fixo de duração de D minutos. Como o tempo é limitado, Marina não conseguirá aproveitar todas as atrações que gostaria.

Para ajudá-la, seu amigo Lucas visitou o parque anteriormente e avaliou cada brinquedo com uma nota de 0 a 10, de acordo com o quanto ele considerou a experiência divertida. Confiando totalmente nas avaliações de Lucas, Marina quer planejar seu dia de forma a maximizar a soma das notas das atrações que irá aproveitar, sem ultrapassar o tempo disponível que ela possui.

Entrada

A primeira linha da entrada consiste em um inteiro T ($0 \leq T \leq 100$) referente ao tempo disponível que ela possui em horas e o inteiro N ($1 \leq N \leq 1000$) indicando a quantidade de brinquedos na lista de Marina. As próximas $N * 2$ linhas contém: uma linha com uma string S ($1 \leq LEN(S) \leq 100$) indicando o nome do brinquedo e na próxima linha: um inteiro I da sua respectiva nota ($0 \leq I \leq 10$) e um inteiro D indicando sua duração em minutos ($0 \leq D \leq 500$).

Saída

A saída deve consistir em um único número inteiro representando a soma máxima das notas dos brinquedos que Lucas conseguiu brincar, considerando o limite de tempo disponível no parque. Caso não seja possível imprima 0.

Exemplo de Entrada 1

<pre> 1 3 Torre de Queda 10 90 Tsunami 9 80 Ciclone 8 75 </pre>	<pre> 0 </pre>
---	------------------------------------

Exemplo de Saída 1

Exemplo de Entrada 2

<pre> 1 3 Elevador Radical 10 20 Treno Radical 8 20 Giroscópio 7 20 </pre>	<pre> 25 </pre>
--	-------------------------------------

Exemplo de Saída 2

Problema K Cocinnitas

Arquivo fonte: concinnitas.{ c | cpp | java | py }
Autor: Prof. Me. Sérgio Luiz Banin (Fatec São Paulo)

Assim como P.C. Fabergé, L.F. Cartier e R.J. Lalique, A.C. Munarovski é um renomado mestre joalheiro, reconhecido mundialmente pelo sucesso de suas peças exclusivas. Entre suas criações mais famosas está a coleção de joias conhecida como Concinnitas, muito apreciada por colecionadores e pessoas de alto padrão socioeconômico.

As três peças Concinnitas mais famosas são:

- Concinnitas Étoile – organizada em uma matriz 59×59 com as gemas em em desenho de estrela;
- Concinnitas Imperium – organizada em uma matriz 60×60 e criada para a coroação do Imperador Alexemberg Marinovitch III;
- Concinnitas Delicata – organizada em uma matriz 19×10 e criada na forma de um broche presenteado à Duquesa GraceAnn de Borghèse;
- Concinnitas Bellina – organizada em uma matriz 3×4 , a menor e mais charmosa peça da coleção.

As peças da coleção Concinnitas possuem uma característica marcante: seu arranjo de gemas forma uma matriz bidimensional com dupla simetria. Isso significa que o arranjo permanece inalterado tanto após uma reflexão horizontal quanto após uma reflexão vertical, em relação a dois eixos perpendiculares alinhados à matriz, conforme ilustrado na Figura 1.

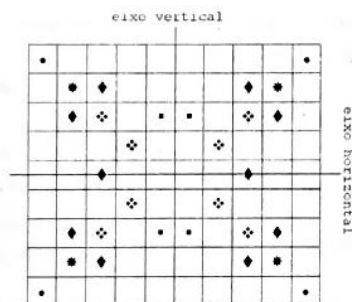


Figura 1

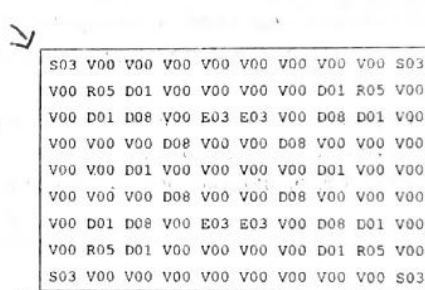


Figura 2

Os mapas das peças podem ser representados por matrizes quadradas ou retangulares. Cada dimensão da matriz possui tamanho mínimo igual a 3. Dependendo das dimensões da matriz, os eixos de simetria podem passar exatamente pelo centro de uma linha ou coluna, ou ainda posicionar-se entre duas linhas ou duas colunas adjacentes.

Outra característica importante é que as gemas utilizadas pertencem exclusivamente aos quatro grupos clássicos de pedras preciosas: diamantes (D), esmeraldas (E), rubis (R) e safiras (S). Além disso, cada gema pode possuir uma lapidação específica.

Cada gema é identificada por um código composto por:

- uma letra representando o tipo da pedra;
- um número de dois dígitos representando a lapidação.

S03
lapid.

Por exemplo:

- D08 representa um diamante com lapidação de forma 8;
- E03 representa uma esmeralda com lapidação de forma 3.

Posições da matriz que não possuem gema são representadas pelo código V00.

Por serem extremamente valiosas, as peças da coleção Concinnitas são frequentemente alvo de falsificações. Durante o processo de avaliação, especialistas identificam as gemas presentes na peça e montam um mapa matricial contendo os códigos correspondentes.

A Figura 2 apresenta o mapa da peça ilustrada na Figura 1.

Sua tarefa é desenvolver um programa capaz de ler o mapa de uma peça e verificar se ele possui a dupla simetria necessária para pertencer à coleção Concinnitas.

Formalmente, considerando uma matriz com N linhas e M colunas, uma peça possui dupla simetria quando cada posição (i, j) contém:

- exatamente o mesmo valor da posição correspondente em relação ao eixo horizontal; e
- exatamente o mesmo valor da posição correspondente em relação ao eixo vertical.

Possuir a dupla simetria não garante que a joia seja autêntica, mas já representa um importante passo no processo de avaliação realizado pelos especialistas.

Entrada

A entrada é constituída de um único caso de teste.

A primeira linha da entrada contém dois inteiros L e A , onde:

- $3 \leq L \leq 60$ representa a largura da matriz.
- $3 \leq A \leq 60$ representa a altura da matriz.

As A linhas seguintes contém L códigos de três caracteres, representando o mapa da matriz.

Saída

A saída é constituída por uma única frase com todas as letras maiúsculas, sem as aspas e seguida do final de linha, da seguinte forma:

- se for identificada a dupla simetria a saída deve ser "PODE SER VERDADEIRA";
- caso contrário a saída deve ser "CERTAMENTE FALSA".

Exemplo de Entrada 1

```

10 9
S03 V00 V00 V00 V00 V00 V00 V00 V00 S03
V00 R05 D01 V00 V00 V00 V00 D01 R05 V00
V00 D01 D08 V00 E03 E03 V00 D08 D01 V00
V00 V00 V00 D08 V00 V00 D08 V00 V00 V00
V00 V00 D01 V00 V00 V00 V00 D01 V00 V00
V00 V00 V00 D08 V00 V00 D08 V00 V00 V00
V00 D01 D08 V00 E03 E03 V00 D08 D01 V00
V00 R05 D01 V00 V00 V00 V00 D01 R05 V00
S03 V00 V00 V00 V00 V00 V00 V00 V00 S03
    
```

Exemplo de Saída 1

PODE SER VERDADEIRA

Exemplo de Entrada 2

```

5 8
E03 V00 V00 V00 E03
V00 V00 D07 V00 V00
V00 D06 V00 D06 V00
V00 R05 V00 R05 V00
V00 V00 E02 V00 V00
V00 D06 V00 D06 V00
V00 V00 D07 V00 V00
E03 V00 V00 V00 E03
    
```

Exemplo de Saída 2

CERTAMENTE FALSA

```

tot = 0
TID = int(input())
N = Brangueador m =
} ENTRADA
S = INPUT()
for motor in i
    IF ( tot D + tot > N )
        Break
    N = N * GO.
    else
        tot +=
for motor in i
    tot += motor
Print ( tot ) if N - D > 0
    
```